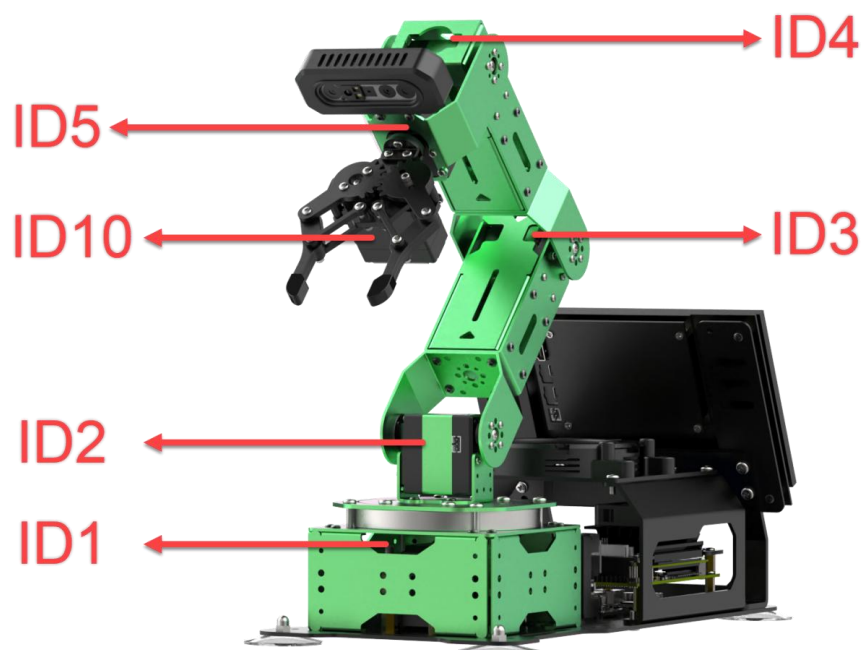


A mechanical arm is an automated device widely used in mechanical technology, finding practical applications in industrial manufacturing, medical treatment, entertainment services, the military, semiconductor manufacturing, and space exploration. Despite having various forms, they all share a common ability to receive instructions and accurately position themselves in three-dimensional (or two-dimensional) space for specific tasks. The functionality of robotic arms is further improved when equipped with vision systems. This document will illustrate the basic structure and motion control of the JetArm ultimate version as an example. The control methods for other versions of robotic arms are also identical.

## 1. Servo Control Node

The robot arm is equipped with 6 servos, utilizes Jetson Nano as the controller, employs an STM32 microcontroller for communication, and implements servo control through the serial port. To facilitate and standardize servo control, it is essential to establish a ROS node as the interface for each servo.



## 1.1 Introduction to Servo Control Node

### 1.1.1 Preparation

- 1) Assemble the robot arm according to the tutorial stored in  
**'1. Basic Lesson /1. JetArm Introductory Manual/3. Accessories Assembly'**.
- 2) Start the robot arm according to the tutorial saved in  
**'1. Basic Lesson /1. JetArm Introductory Manual/4. Initial Startup and Test'**.
- 3) Access the robot arm system desktop according to the tutorials locating in  
**'1. Basic Lesson /1. JetArm Introductory Manual/11. Remote Desktop System Installation and Connection'**.

### 1.1.2 Initiate Servo Control Node



- 1) Click on  to open the terminal, then execute the command '**sudo systemctl stop jetarm\_bringup.service**' in the terminal and press Enter. Enter the password '**hiwonder**' to terminate the auto-start service. The password will be hidden as you input it, and press Enter once the input is complete.

```
> sudo systemctl stop jetarm_bringup.service  
[sudo] password for hiwonder:
```

- 2) Run the command '**roslaunch jetarm\_sdk sdk\_node\_6dof.launch**' to initiate the robot arm SDK file.

```
> roslaunch jetarm_sdk sdk_node_6dof.launch
```

- 3) Open a new terminal and run the command '**rostopic list**' to display the topics published after the initiation of the robot arm node.

```
> rostopic list
/controllers/id_pos_dur
/controllers/multi_id_pos_dur
/controllers/set_joint
/controllers/set_servo
/jetarm_sdk/serial_servo/load_unload
/jetarm_sdk/serial_servo/move
/jetarm_sdk/serial_servo/stop
/jetarm_sdk/set_buzzer
/jetarm_sdk/set_led
/joint_states
/rosout
```

| Topic                                | Function                 |
|--------------------------------------|--------------------------|
| /jetarm_sdk/serial_servo/load_unload | Power on/off servos      |
| /jetarm_sdk/serial_servo/move        | Control servo movements  |
| /jetarm_sdk/serial_servo/stop        | Terminate servo movement |

## 1.2 Servo Control Through Topic

### 1.2.1 Terminal Control

- 1) Open a new terminal, and execute the command '**rostopic pub**  
**/jetarm\_sdk/serial\_servo/move**' and hit 'Tab' key to auto-complement the command.

```
> rostopic pub /jetarm_sdk/serial_servo/move hiwonder_interfaces/SerialServoMove
"servo_id: 0
position: 0
duration: 0"
```

- 2) The program will print the following three parameters:

**servo\_id:** Adjustable from 1 to 5.

**Position:** Servo angle, ranging from 0 to 1000.

**Duration:** Time required for the servo to move to the specified position, measured in milliseconds.

For example, in this program, the N0.1 servo is instructed to rotate to an angle of 500 in 1000 milliseconds.

```
> rostopic pub /jetarm_sdk/serial_servo/move hiwonder_interfaces/SerialServoMove
"servo_id: 1
position: 500
duration: 1000"
publishing and latching message. Press ctrl-C to terminate
```

## 1.2.2 Python Control

- 1) Run the command '**cd jetarm/src/jetarm\_example/src/bus\_servo/**' to navigate to the corresponding directory.

```
> cd jetarm/src/jetarm_example/src/bus_servo
```

- 2) Execute the command '**python3 bus\_servo.py**' to initiate the game. Then N0.4 servo will rotate within the range of 200-500, and the servo topic information will be printed on the terminal.

```
> python3 bus_servo.py
servo_id: 4
position: 200
duration: 500
servo_id: 4
position: 500
duration: 500
```

## 1.2.3 Launch File Control

- 1) Use the shortcut '**Ctrl+C**' to terminate the execution of the launch file in the terminal running the robot arm SDK.
- 2) Execute the command '**roslaunch jetarm\_example bus\_servo.launch**' to initiate the servo control. The **buservo.py** file will be invoked.

```
> roslaunch jetarm_example bus_servo.launch
```

- 3) After initiating the game, the No. 4 servo will rotate back and forth within the range of 200-500, and the servo topic information will be printed on the terminal.

```
[INFO] [1693831680.184304]: USE_SIM is False
[INFO] [1693831680.207227]: Detected serial servo ID:2 for joint2 and added
[INFO] [1693831680.254560]: Detected serial servo ID:3 for joint3 and added
[INFO] [1693831680.312421]: Detected serial servo ID:1 for joint1 and added
[INFO] [1693831680.384748]: Detected serial servo ID:4 for joint4 and added
[INFO] [1693831680.434548]: Detected serial servo ID:5 for joint5 and added
[INFO] [1693831680.502469]: Detected serial servo ID:10 for r_joint and added
servo_id: 4
position: 200
duration: 500
[INFO] [1693831680.777659]: SDK node is running ...
servo_id: 4
position: 500
duration: 500
servo_id: 4
position: 200
duration: 500
```

- 4) If you want to terminate the game, use short-cut '**Ctrl+C**'. If the game cannot be stopped, please have a retry.

### 1.3 Initiate App Auto-Start Service

- 1) After experiencing the game above, it is necessary to initiate the app service; otherwise, the robot's app function will be affected. Execute the command '**sudo systemctl start jetarm\_bringup.service**' and hit Enter to start the app service, then enter the password '**hiwonder**'.

```
> sudo systemctl start jetarm_bringup.service
[sudo] password for hiwonder:
>
```

- 2) The robot arm will return to the initial pose once the app service is initiated, and the buzzer will emit a '**beep**' sound.

### 1.4 Program Analysis

#### 1.4.1 Launch File Analysis

The launch file locates in this folder,

**'/home/hiwonder/jetarm/src/jetarm\_example/src/bus\_servo/bus\_servo.launch'**.

'node\_name' defines the node name

```
3 <arg name="node_name" default="bus_servo"/>
```

Initiate the 'sdk\_node\_6dof.launch' file and invoke the robot arm hardware control node.

```
5 <include file="$(find jetarm_sdk)/launch/sdk_node_6dof.launch"/>
```

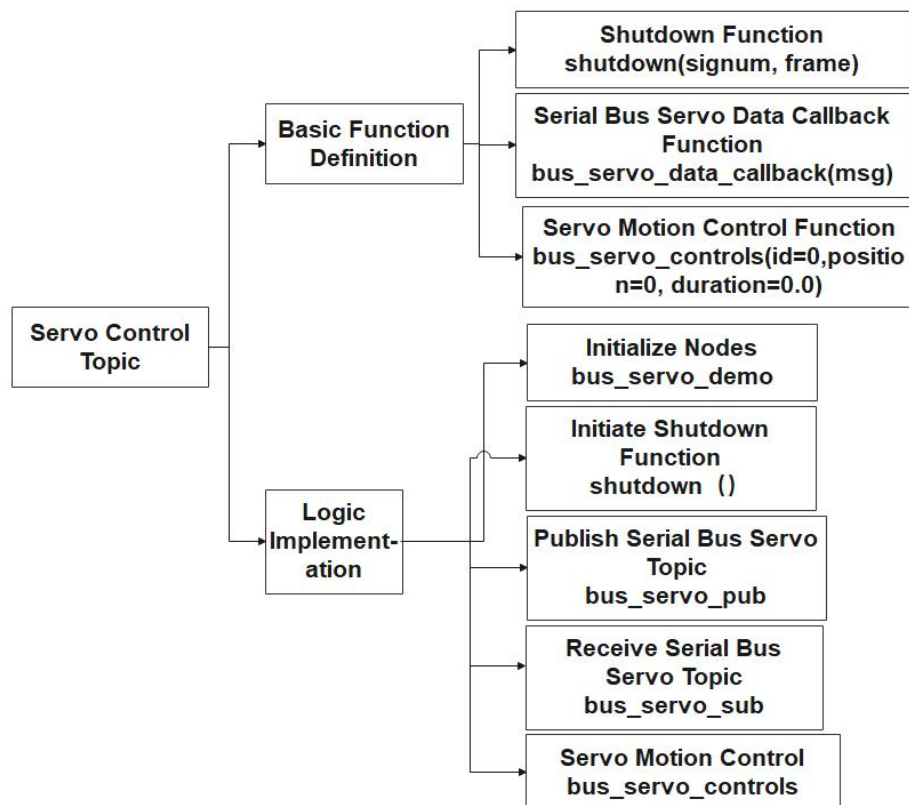
Execute the source code file.

```
6 <node pkg="jetarm_example" type="bus_servo.py" name="$(arg node_name)" output="screen">
7 </node>
```

## 1.4.2 Python Program Analysis

The source code of the program is saved in '/home/hiwonder/jetarm/src /jetarm\_example/src/bus\_servo/bus\_servo.py'.

The program logic flowchart is as below:





The diagram above illustrates the main structure of the program's logic flow, highlighting the division into basic function definitions and implementation logic. Initially, the necessary functions for servo control are established. In the main function, nodes are initialized, interrupt functions are activated to detect signals, and relevant bus servo topics are both published and received. This information is then relayed to the servo control function, enabling effective servo control. The upcoming document will follow the structure outlined in the program's logic flowchart above.

## ◆ Import Feature Package

By utilizing the import statement to bring in necessary modules, the rospy library serves as the Python client library for ROS. This library is essential for tasks such as crafting ROS nodes, managing message publication and subscription, and executing service calls. The 'signal' library, a standard Python module, is employed for signal handling. Furthermore, the **hiwonder\_interfaces.msg** package encapsulates various message types, including the **SerialServoMove** message type specifically designed for serial communication with the robotic arm.

```
5 import rospy
6 import signal
7 from hiwonder_interfaces.msg import SerialServoMove
```

## ◆ Main Function

```
39 if __name__ == '__main__':
40     rospy.init_node('bus_servo_demo', anonymous=True) #初始化节点
41     signal.signal(signal.SIGINT, shutdown)
42     # 发布总线舵机话题
43     bus_servo_pub = rospy.Publisher('/jetarm_sdk/serial_servo/move', SerialServoMove, queue_size=1)
44     # 接收总线舵机话题
45     bus_servo_sub = rospy.Subscriber('/jetarm_sdk/serial_servo/move', SerialServoMove,
46     bus_servo_data_callback)
47     while running:
48         bus_servo_controls(id =4,position =500,duration=500) #发布数据
49         rospy.sleep(0.25) # 运行时间
50         bus_servo_controls(id =4,position =200,duration=500) #发布数据
51         rospy.sleep(0.25) # 运行时间
```

Initialize the node.

```
40 if __name__ == '__main__':
41     rospy.init_node('bus_servo_demo', anonymous=True) #初始化节点
```

Establish the signal handling function, triggering the '**shutdown**' function upon the reception of an interrupt signal (Ctrl+C).

```
41 signal.signal(signal.SIGINT, shutdown)
```

Publish and receive the serial bus servo topic, triggering the '**bus\_servo\_data\_callback**' function upon receipt of the serial bus servo topic.

```
42 # 发布总线舵机话题
43 bus_servo_pub = rospy.Publisher('/jetarm_sdk/serial_servo/move', SerialServoMove, queue_size=1)
44 # 接收总线舵机话题
45 bus_servo_sub = rospy.Subscriber('/jetarm_sdk/serial_servo/move', SerialServoMove, bus_servo_data_callback)
```

When the '**running**' variable is set to True, the loop will run continuously. Inside the loop, the angle of servo 4 is first set to 500, maintaining this position for 500 milliseconds with an execution time of 0.25 seconds for this action. Afterward, the angle of servo 4 is set to 200, once again running for 500 milliseconds. These two actions are alternately executed in a continuous manner. Modifying the servo ID, servo angle, and runtime within the loop allows for achieving different effects.

```
46 while running:
47     bus_servo_controls(id=4, position=500, duration=500) #发布数据
48     rospy.sleep(0.25) # 运行时间
49     bus_servo_controls(id=4, position=200, duration=500) #发布数据
50     rospy.sleep(0.25) # 运行时间
51
```

## ◆ self.shutdown Function

Interrupt function within the program, intended to stop its execution. When the program receives an interrupt signal, it changes the '**running**' status to '**False**' and prints '**shutdown**' upon completion.

```
14 def shutdown(signum, frame):
15     global running
16     running = False
17     rospy.loginfo('shutdown')
18     rospy.signal_shutdown('shutdown')
19
```

## ◆ Serial Bus Servo Data Callback Function



Establish a callback function to manage received messages. The function, with a parameter '**msg**' representing the received message, declares '**bus\_servo\_data\_detection**' as a global variable. Within the function, upon receiving a message, it verifies if the '**servo\_id**' in the message is non-empty. If it is not empty, '**bus\_servo\_data\_detection**' is then set to True. This mechanism is employed to ascertain the presence of valid data in the received message.

```
20 # 总线舵机数据回调函数
21 def bus_servo_data_callback(msg):
22     global bus_servo_data_detection
23     print(msg)
24     if msg.state[0].servo_id != []: #判断该话题的ID是否为空
25         bus_servo_data_detection = True
```

### ◆ Servo Motion Control Function

**def bus\_servo\_controls(id=0, position=0, duration=0.0)** is used to control serial bus servo's movement. It contains the following parameters:

**ID:** Servo ID; Adjustable from 1 to 5.

**Position:** Servo angle, ranging from 0 to 1000.

**Duration:** Time required for the servo to move to the specified position, measured in milliseconds.

```
27 def bus_servo_controls(id=0,
28                         position=0,
29                         duration=0.0):
30
31     # 设置总线舵机消息类型
32     data = SerialServoMove()
33     data.servo_id = id #总线舵机ID
34     data.position = position #总线舵机角度[0-1000]
35     data.duration = duration #总线舵机运行时间
36     bus_servo_pub.publish(data) #发布数据
37
```

## 2. LED Control Node

STM32 controller includes a LED. The STM32 control board is fitted with an LED. Using the Jetson Nano as the main controller, we establish communication with the STM32 microcontroller to control the LED through

serial communication. For convenient and standardized LED control, it is essential to set up a ROS node as the interface.